Métodos de Ordenação Híbrida Adaptativa

Um Estudo Comparativo: Timsort e Introsort

Italo Xavier Fonseca João Paulo de Abreu Duarte Matheus Tavares de Bessas Thales Guerreiro

CEFET-MG - Engenharia de Computação
Campus V - Divinópolis

Maio de 2026

Sumário Geral

1. Introdução
2. Fundamentação Teórica
3. Metodologia
4. Modelagem Experimental
5. Resultados
6. Conclusão
7. Conclusão
8. Referências

Sumário

1. Introdução
2. Fundamentação Teórica
3. Metodologia
4. Modelagem Experimental
5. Resultados
6. Conclusão
7. Conclusão
8. Referências

1.1 Contextualização

O Problema da Ordenação

A evolução das linguagens de programação exigiu algoritmos de ordenação mais robustos. Diferente dos algoritmos puramente teóricos, as implementações modernas buscam o equilíbrio entre eficiência computacional e previsibilidade.

- **Timsort (2002):** Criado por Tim Peters. Observa que dados do mundo real frequentemente possuem subsequências já ordenadas. Une a estabilidade do Merge Sort com a baixa sobrecarga do Insertion Sort.
- **Introsort (1997):** Criado por David Musser. Soluciona a degradação quadrática do Quicksort, alternando dinamicamente para o Heapsort ao atingir uma profundidade crítica de recursão.

1.2 Justificativa da Relevância

- **1.2.1 Relevância do Timsort:** A sua característica estável e adaptativa é fundamental em grandes volumes de dados (ex: MovieLens) para preservar a ordem relativa de registos idênticos. É o padrão em Python, Java e Swift.
- **1.2.2 Relevância do Introsort:** Justifica-se pela sua eficiência de espaço (*in-place*) e garantia de pior caso. Essencial em sistemas de baixo nível (como a STL do C++) e ambientes com memória restrita, onde a economia de RAM compensa a perda de estabilidade.

Sumário

1. Introdução
2. Fundamentação Teórica
3. Metodologia
4. Modelagem Experimental
5. Resultados
6. Conclusão
7. Conclusão
8. Referências

2.1 Conceitos Formais e 2.2 Complexidade

- **Problema Formal:** Encontrar uma permutação $A' = \{a'_1, a'_2, ..., a'_n\}$ tal que $a'_1 \leq a'_2 \leq \cdots \leq a'_n$. Algoritmos baseados em comparação possuem limite inferior de $\Omega(n \log n)$.

Algoritmo	Melhor Caso	Médio Caso	Pior Caso
Timsort	$O(n)$	$O(n \log n)$	$O(n \log n)$
Introsort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

Tabela: Comparação de Complexidade Assintótica

Nota: O Timsort atinge $O(n)$ em dados parcialmente ordenados, evidenciando a sua alta adaptatividade.

2.3 Propriedades Relevantes

2.3.1 Estabilidade

- **Timsort:** Estável (preserva chaves idênticas).
- **Introsort:** Instável (trocas alteram a ordem relativa).

2.3.2 In-place

- **Timsort:** Não é in-place. Requer memória auxiliar $O(n)$ para o *merge*.
- **Introsort:** É in-place. Utiliza apenas a pilha de recursão.

2.3.3 Adaptatividade

- **Timsort:** Altamente adaptativo (identifica *runs* naturais).
- **Introsort:** Não adaptativo (processamento uniforme).

Sumário

1. Introdução
2. Fundamentação Teórica
3. Metodologia
4. Modelagem Experimental
5. Resultados
6. Conclusão
7. Conclusão
8. Referências

3.1 Descrição dos Algoritmos

A Mecânica do Timsort:

1. Identifica sequências crescentes/decrescentes (*runs*).
2. Usa Insertion Sort se a *run* for menor que o limite *minrun*.
3. Uma pilha de fusão garante mesclagem balanceada (*Merge*).

A Mecânica do Introsort:

1. Particionamento Dinâmico: Inicia com Quicksort (pivô via mediana de três).
2. *Fallback*: Alterna para Heapsort se a recursão ultrapassar $2 \log_2 n$.
3. Otimização: Finaliza partições pequenas (geralmente < 16) com Insertion Sort, otimizando a cache.

3.2 Pseudocódigo: Timsort

Algoritmo 1: Lógica de Execução

```
Procedure Timsort(arr, n):
  minRun = CalcularMinRun(n)

  // Fase 1: Ordenar pequenas partições (runs)
  para i = 0 até n-1 step minRun faça:
    InsertionSort(arr, i, min(i + minRun - 1, n - 1))

  // Fase 2: Mesclar runs ordenadas (Merge)
  size = minRun
  enquanto size < n faça:
    para left = 0 até n-1 step 2 * size faça:
      mid = left + size - 1
      right = min(left + 2 * size - 1, n - 1)
      se mid < right então:
        Merge(arr, left, mid, right)
    size = 2 * size
```

3.2 Pseudocódigo: Introsort

Algoritmo 2: Procedimento com Fallback Automático

```
Procedure Introsort(arr, n):
  depthLimit = 2 * log2(n)
  IntroUtil(arr, 0, n - 1, depthLimit)

Function IntroUtil(arr, low, high, depthLimit):
  size = high - low + 1
  se size < 16 então:
    InsertionSort(arr, low, high) // Otimização L1 Cache
    retorna

  se depthLimit == 0 então:
    Heapsort(arr, low, high) // Proteção contra O(n^2)
    retorna

  p = Partition(arr, low, high) // Quicksort normal
  IntroUtil(arr, low, p, depthLimit - 1)
  IntroUtil(arr, p + 1, high, depthLimit - 1)
```

Sumário

1. Introdução
2. Fundamentação Teórica
3. Metodologia
4. Modelagem Experimental
5. Resultados
6. Conclusão
7. Conclusão
8. Referências

4.1 Ambiente Computacional

Para garantir a integridade das medições, todos os testes rodaram num ambiente Linux restrito.

Hardware

- **Processador:** Intel Core i5-2450M (2 núcleos, 4 *threads*, 2.50GHz)
- **Memória RAM:** 8GB (2x4GB) DDR3 1600MHz (Dual Channel)
- **Armazenamento:** SSD SATA III de 512GB

Software e Compiladores

- **Sistema Operativo:** Arch Linux (Kernel 6.x)
- **Ferramentas:** GCC 13.2.1, Python 3.12.1, .NET SDK 8.0, GnuCOBOL 3.2.

4.2 Linguagens e 4.3 Experimentos

As 5 Linguagens Utilizadas

- **C e C++:** Base da computação de alto desempenho e controlo *bare-metal*.
- **C#:** Ambiente gerido (CLR), equilíbrio produtividade/performance.
- **Python:** Linha de base adaptativa e berço nativo do Timsort.
- **Cobol:** *Diferencial do estudo*. Inserido para avaliar o impacto estrutural de um ambiente legado.

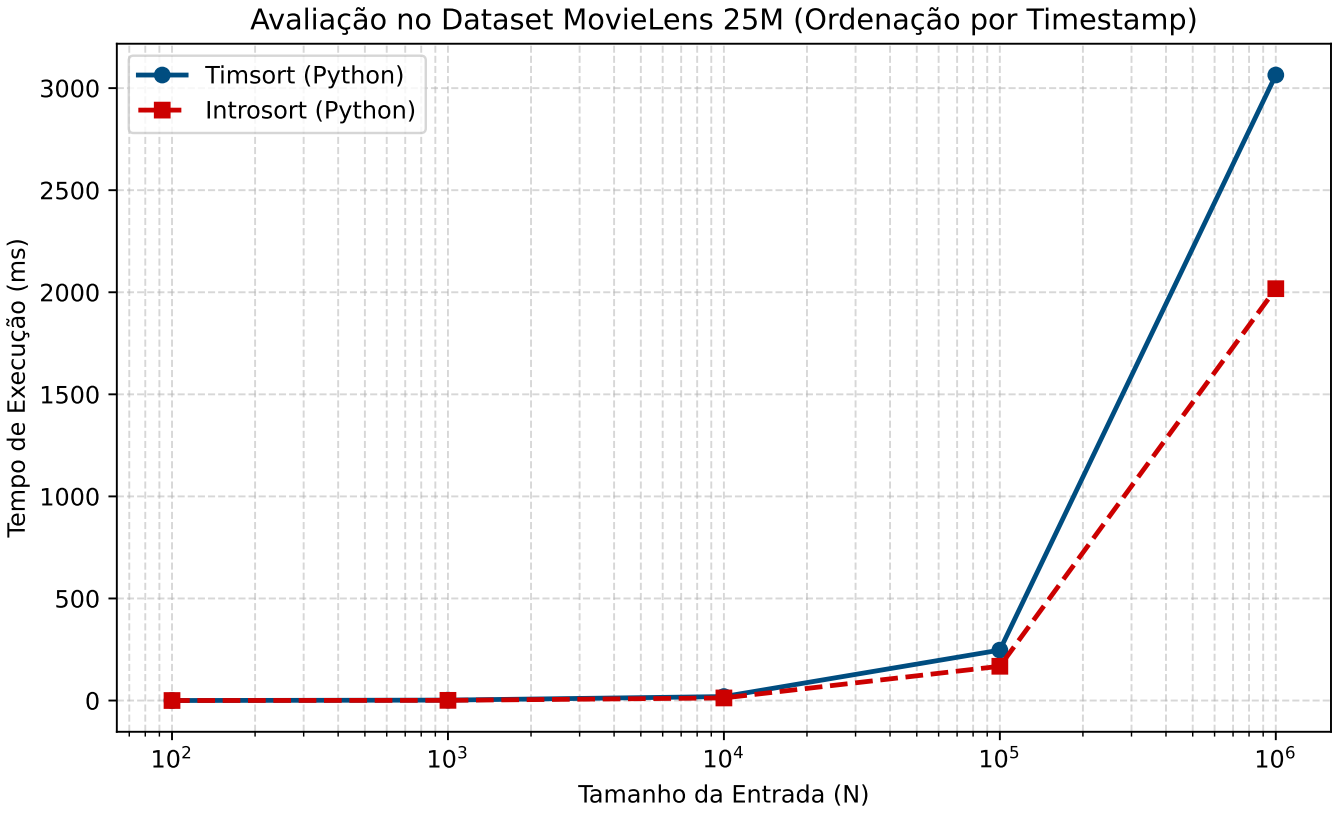
Bases de Dados dos Testes

1. **Dataset Analítico:** MovieLens 25M (`ratings.csv` - *Timestamp*).
2. **Datasets de Esforço:** Aleatório, Ordenado e Invertido (10^2 a 10^6 elementos).

Sumário

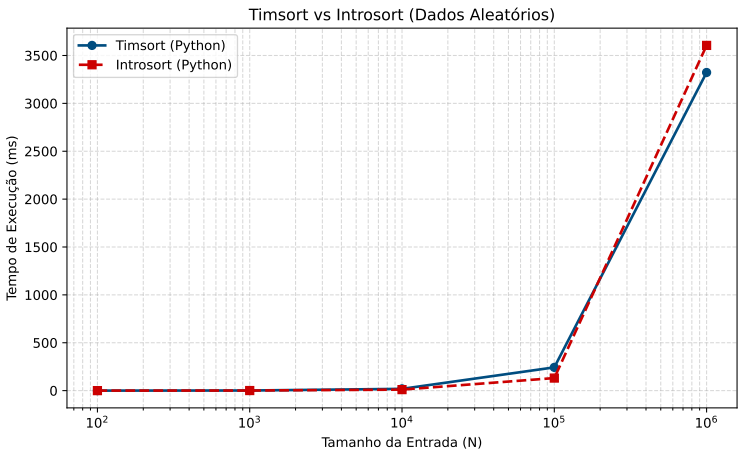
1. Introdução
2. Fundamentação Teórica
3. Metodologia
4. Modelagem Experimental
5. Resultados
6. Conclusão
7. Conclusão
8. Referências

5.1 Avaliação com Dataset MovieLens 25M

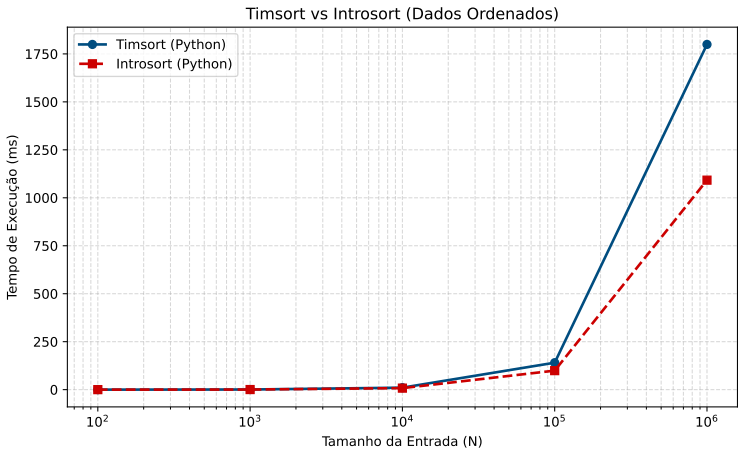


- **Cenário:** Ordenação cronológica pelas *timestamps* de avaliações.
- **A Anomalia:** O Introsort superou o Timsort na marca de 10^6 elementos.
- **Justificação:** A exigência de memória $O(n)$ do Timsort penalizou a linguagem Python, que sofreu com *overhead* de alocação de memória, enquanto o Introsort operou *in-place*.

5.2 Testes com Entradas Crescentes (Aleatório e Ordenado)

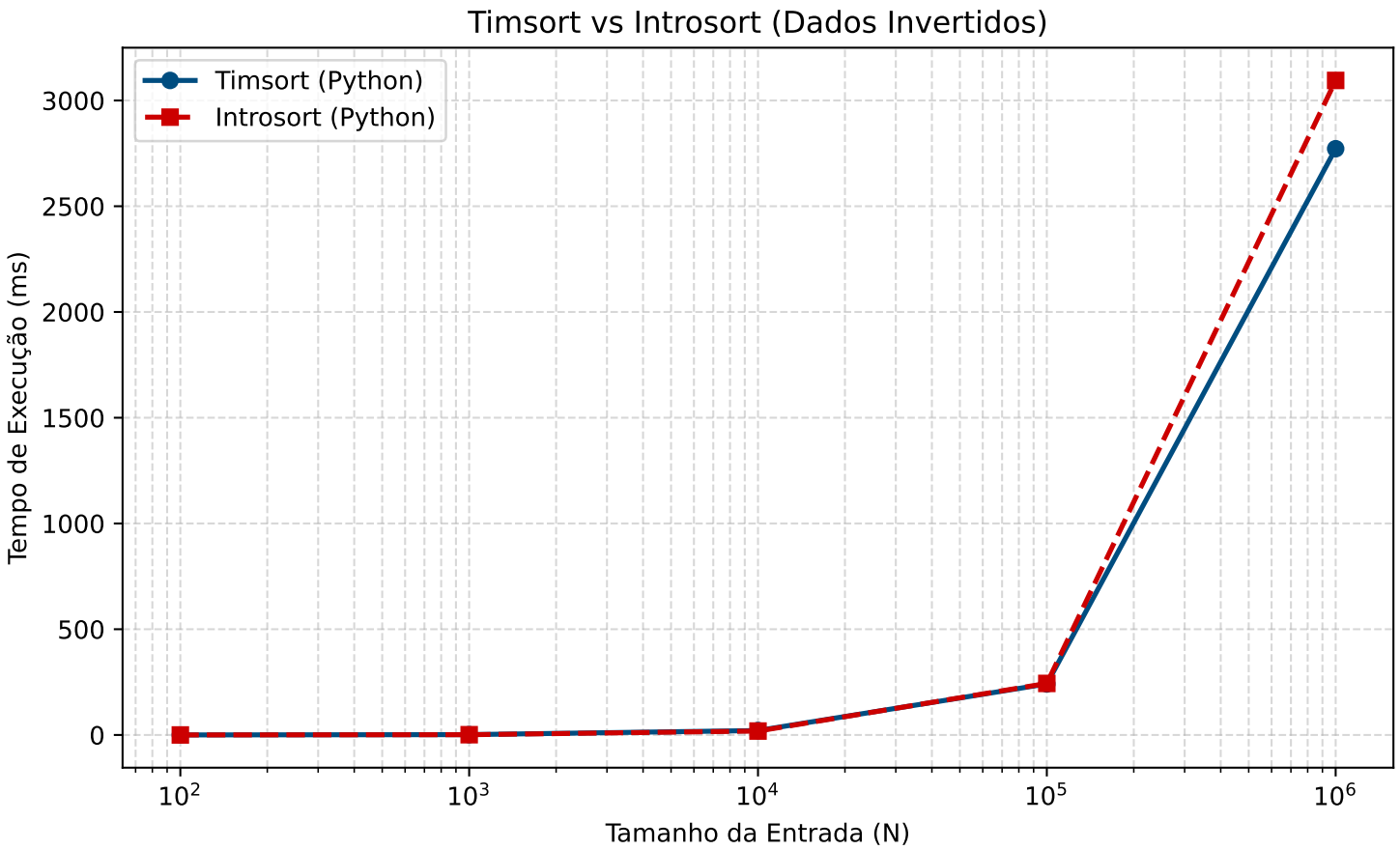


Aleatório: Empate técnico até 10^5. Escalabilidade linear-logarítmica similar.



Ordenado: Demonstração teórica da adaptatividade do Timsort no seu melhor caso $O(n)$, até esbarrar nos limites da linguagem.

5.2 Testes com Entradas Crescentes (Invertido)



- **Cenário:** Pior distribuição possível (Ordem decrescente).
- **Resultado:** O Timsort é altamente otimizado para reverter blocos decrescentes inteiros e fundi-los, sendo sensivelmente mais rápido do que as particões do Introsort nesta disposição adversa.

5.4 Análise dos Resultados e Hardware

O Impacto do Hardware e da Linguagem

- **Restrição de RAM (8GB):** A propriedade *in-place* do Introsort revelou-se crucial para processar volumes massivos sem degradação do sistema.
- **O Custo das Abstrações (Python):** Algoritmos complexos como o Timsort, que exigem alocação dinâmica contínua (para fatiar listas e fazer o *merge*), são severamente penalizados por interpretadores. A mesma implementação em C++ apresentaria uma vantagem muito superior do Timsort.

Sumário

1. Introdução
2. Fundamentação Teórica
3. Metodologia
4. Modelagem Experimental
5. Resultados
6. Conclusão
7. Conclusão
8. Referências

Sumário

1. Introdução
2. Fundamentação Teórica
3. Metodologia
4. Modelagem Experimental
5. Resultados
6. Conclusão
7. Conclusão
8. Referências

6.1 Síntese dos Achados

- **Desempenho Bruto:** C e C++ mantêm-se como o padrão de referência (tempos sub-segundo para 10^6 elementos). A linguagem C# demonstrou forte competitividade devido às otimizações do compilador JIT.
- **Adaptatividade:** O Timsort confirmou a sua superioridade em dados parcialmente ordenados, reduzindo drasticamente o tempo de execução nestes cenários.
- **Custo de Abstração:** O Python apresentou um consumo de memória até dez vezes superior em comparação às linguagens estaticamente tipadas, evidenciando o *trade-off* entre facilidade de desenvolvimento e eficiência de hardware.

6.2 Limitações do Estudo

- **Uniformidade dos Dados:** Os testes foram restritos a vetores de números inteiros, não explorando o desempenho (custo de comparação) em tipos de dados complexos ou *strings*.
- **Ambiente de Execução Fixo:** Os testes foram realizados numa única arquitetura *desktop*. Variações na hierarquia de cache do processador podem influenciar os tempos de resposta de forma diferente.
- **Escopo Algorítmico:** A análise limitou-se às implementações canónicas e sequenciais, não considerando variantes paralelas ou otimizações específicas de baixo nível (como instruções SIMD).

6.3 Propostas de Trabalhos Futuros

- **Ordenação em Larga Escala:** Expandir os testes para além dos limites da memória RAM, avaliando algoritmos de ordenação externa e o impacto da latência de disco/SSD.
- **Análise de *Cache Misses*:** Utilizar ferramentas de *profiling* de hardware para quantificar o impacto dos erros de cache no desempenho do Introsort versus Timsort em diferentes arquiteturas.
- **Processamento Paralelo:** Investigar o desempenho de versões *multithreaded* e implementações em GPU (via CUDA ou OpenCL), comparando-as com as versões sequenciais.

Sumário

1. Introdução
2. Fundamentação Teórica
3. Metodologia
4. Modelagem Experimental
5. Resultados
6. Conclusão
7. Conclusão
8. Referências

Referências

- MUSSER, David. *Introspective Sorting and Selection Algorithms*. Software: Practice and Experience, 1997.
- GEEKSFORGEEKS. *Introsort or Introspective Sort*. Disponível em: <https://www.geeksforgeeks.org>.
- UEPG. *REA - TimSort*. Material Académico, 2019. Disponível em: <https://deinfo.uepg.br>.

Obrigado pela atenção!

Questões ou comentários?